

Jarbas

Desenvolvimento de um Assistente de Voz para Home Assistant com
Flutter, Dart e Vosk

Relatório Técnico em Formato de Tutorial

Projeto ha_voice_app ("Jarbas") — github.com/Beto-Inovacao-LTDA/jarbas

Setembro de 2026

Sumário

1. Introdução	4
2. Por que Flutter para Android (e, no futuro, iOS)	5
3. Introdução ao Flutter: anatomia de uma aplicação	6
3.1 “Tudo é um widget”	6
3.2 Widgets com e sem estado: StatelessWidget e StatefulWidget	6
3.3 Anatomia de um projeto Flutter	6
3.4 O ciclo de widgets raiz: MaterialApp e Scaffold	7
4. Introdução à linguagem Dart	8
4.1 Conceitos essenciais usados no Jarbas	8
4.2 Bons tutoriais em português	8
5. Por que o Jarbas é um bom estudo de caso de Flutter	10
5.1 Aplicação dos principais widgets do Flutter	10
5.2 Uso de comunicação em rede	10
5.3 Uso de recursos avançados de hardware	10
6. Configurações de rede e segurança no Android	12
6.1 Permissões no AndroidManifest.xml	12
6.2 Network Security Configuration e tráfego HTTP sem TLS	12
7. HTTP, APIs REST e autenticação por Bearer Token	14
7.1 Por que REST sobre HTTP	14
7.2 O contrato HTTP usado pelo Jarbas	14
7.3 Tratamento defensivo de erros de rede	15
8. Home Assistant: contexto para leigos	16
9. A escolha do Vosk em detrimento do Picovoice	17
9.1 Tentativa inicial: Picovoice / Porcupine	17

9.2 Escolha final: Vosk	17
9.3 Comparativo	17
9.4 Nota técnica: escolha do pacote Flutter para o Vosk	18
10. Anatomia detalhada do código	19
10.1 pubspec.yaml — manifesto do pacote	19
10.2 lib/main.dart — ponto de entrada	19
10.3 lib/ha_service.dart — cliente HTTP do Home Assistant	20
10.4 lib/home_screen.dart — tela principal	21
10.5 lib/settings_store.dart — persistência local	22
10.6 lib/jarbas_service.dart — Modo Jarbas (wake word e serviço)	23
10.7 lib/settings_screen.dart — tela de configurações	25
10.8 android/app/src/main/AndroidManifest.xml	25
10.9 android/app/build.gradle.kts	26
10.10 android/app/src/main/res/xml/network_security_config.xml	26
10.11 android/app/proguard-rules.pro	26
11. Conclusão	28
12. Referências	29

1. Introdução

Este relatório documenta, em formato de tutorial, as decisões técnicas e a implementação do **Jarbas** (nome interno do pacote: `ha_voice_app`), um aplicativo Android — com portabilidade planejada para iOS — que funciona como um assistente de voz local para o [Home Assistant](#), a plataforma open-source de automação residencial. O app tem dois modos de uso: *sob demanda* (o usuário toca em um botão de microfone e fala um comando) e o *Modo Jarbas*, em que o aparelho fica continuamente ouvindo, offline, por uma palavra de ativação (“OK Jarbas”), de forma semelhante a uma Amazon Echo ou Google Home — rodando em segundo plano em um celular Android antigo dedicado.

O objetivo deste documento é duplo. Primeiro, servir como **tutorial introdutório** a quem nunca programou em Flutter/Dart, cobrindo os fundamentos necessários para entender o código do projeto. Segundo, servir como **relatório técnico** que justifica cada decisão de arquitetura relevante — por que Flutter, por que a aplicação escolhida é um bom estudo de caso, por que Vosk no lugar de Picovoice, como a rede e a segurança foram configuradas, e como cada arquivo do repositório funciona. Todas as afirmações técnicas de caráter geral (não específicas deste projeto) trazem uma referência linkável para a documentação oficial correspondente.

Como ler este documento: os Capítulos 2 a 4 são a base conceitual (Flutter, anatomia de um app Flutter, e a linguagem Dart) — podem ser pulados por quem já conhece o ecossistema. Os Capítulos 5 a 9 justificam as escolhas de arquitetura do Jarbas especificamente. O Capítulo 10 é uma referência de código, arquivo por arquivo, para quem for manter ou estender o projeto.

2. Por que Flutter para Android (e, no futuro, iOS)

O [Flutter](#) é um framework de código aberto mantido pelo Google para construir aplicações compiladas nativamente a partir de uma única base de código-fonte, em Dart, para múltiplas plataformas: Android, iOS, web, Windows, macOS e Linux. A escolha do Flutter para o Jarbas foi motivada por quatro fatores concretos:

- **Um único código-fonte para Android e iOS.** O app foi desenvolvido e validado inicialmente no Android (onde vive o principal caso de uso — um celular Android antigo dedicado como “assistente de mesa”), mas a mesma base em [lib/](#) compila para iOS sem reescrita de lógica de negócio. A camada que muda entre plataformas é a de plugins nativos (ex.: reconhecimento de voz, serviço em primeiro plano), e o Flutter isola essa diferença atrás de uma API Dart única — ver o guia oficial de [build para iOS](#).
- **Compilação nativa, não WebView.** Diferente de frameworks híbridos baseados em WebView, o Flutter compila para código de máquina ARM (Android/iOS) e desenha a interface com sua própria engine gráfica (Skia/Impeller), renderizando a 60fps/120fps sem depender do conjunto de widgets nativos da plataforma. Isso é particularmente relevante para o Modo Jarbas, que precisa manter um serviço de áudio responsivo em segundo plano.
- **Hot reload e produtividade.** O ciclo editar-compilar-testar em um aparelho físico caiu de minutos para segundos durante o desenvolvimento (ver [Flutter: hot reload](#)), o que foi decisivo neste projeto porque grande parte da validação exigiu testes iterativos em um Galaxy Note 9 real (não em emulador), incluindo a depuração de dois crashes nativos do motor de reconhecimento de voz.
- **Ecossistema maduro de pacotes via pub.dev.** Todas as integrações de hardware usadas no Jarbas — microfone/reconhecimento de voz, vibração, áudio, serviço em primeiro plano, permissões — vêm de pacotes publicados em [pub.dev](#), o repositório central de pacotes Dart/Flutter, evitando escrever código nativo Kotlin/Swift à mão para a maior parte da aplicação.

É importante registrar a contrapartida dessa escolha: pacotes com binário nativo (plugins) ainda dependem da qualidade de manutenção de terceiros — um tema que reaparece nos Capítulos 5.3 e 9, quando o projeto teve que trocar de pacote de reconhecimento de wake word por incompatibilidade de versão do Android SDK.

3. Introdução ao Flutter: anatomia de uma aplicação

Este capítulo é um tutorial rápido para quem nunca abriu um projeto Flutter. A referência canônica e mais completa é a [Visão Arquitetural do Flutter](#) na documentação oficial.

3.1 “Tudo é um widget”

A ideia central do Flutter é que praticamente todo elemento de interface — texto, ícone, botão, mas também espaçamento, alinhamento, grade e até o próprio tema visual do app — é um **widget**: uma classe Dart imutável que descreve “como deveria ser” um pedaço da tela em função do seu estado atual (ver [Introduction to widgets](#)). Widgets são compostos em árvore: um widget pai recebe outros widgets como parâmetro (o famoso parâmetro `child` ou `children`) e assim se constrói toda a tela, camada sobre camada.

Internamente o Flutter mantém três árvores paralelas — a árvore de *widgets* (configuração imutável), a árvore de *elements* (instâncias vivas que fazem a ponte entre widgets e a árvore seguinte) e a árvore de *RenderObjects* (responsável por medir, posicionar e pintar de fato na tela). Essa separação é o que permite ao Flutter reconstruir só as partes da UI que mudaram, em vez da tela inteira, a cada atualização de estado.

3.2 Widgets com e sem estado: StatelessWidget e StatefulWidget

Um `StatelessWidget` descreve uma UI que não muda sozinha — só é reconstruída se o widget pai mudar seus parâmetros (é o caso de `MyApp` em `lib/main.dart`, no Jarbas). Um `StatefulWidget` tem um objeto `State` associado, que pode guardar dados mutáveis e chamar `setState()` para pedir ao framework que reconstrua a tela com os novos valores. Todas as três telas do Jarbas (`HomeScreen`, `SettingsScreen`) ou seus equivalentes de serviço em segundo plano usam esse padrão: veja, por exemplo, como `_HomeScreenState` chama `setState(() => _micState = _MicState.listening)` ao iniciar a escuta do microfone (Capítulo 10.4).

3.3 Anatomia de um projeto Flutter

Um projeto Flutter recém-criado (via `flutter create`) gera a seguinte estrutura, que se mantém no Jarbas:

Estrutura de diretórios de um projeto Flutter (igual à do repositório jarbas).

meu_app/	
■■■ lib/	# código Dart da aplicação (o que este relatório documenta)
■ ■■■ main.dart	# ponto de entrada – função main()
■ ■■■ android/	# projeto Android nativo (Gradle, Manifest, recursos)
■ ■■■ app/src/main/	
■ ■■■ AndroidManifest.xml	
■ ■■■ res/xml/network_security_config.xml	
■■■ ios/	# projeto Xcode (build para iOS)
■■■ test/	# testes automatizados (widget tests, unit tests)
■■■ assets/	# arquivos estáticos (imagens, sons, modelos de IA, ...)
■■■ pubspec.yaml	# manifesto do pacote: dependências, assets, metadados
■■■ pubspec.lock	# versões exatas resolvidas das dependências

O arquivo mais importante do ponto de vista de configuração é o `pubspec.yaml`, análogo ao `package.json` do Node.js ou ao `pom.xml` do Maven: declara nome, versão, SDK mínimo, dependências (pacotes de [pub.dev](#)) e quais arquivos de `assets/` devem ser empacotados dentro do aplicativo final. O Capítulo 10.1 detalha o `pubspec.yaml` real do Jarbas.

A pasta `android/` é um projeto Android/Gradle completo e convencional — é nela que vivem o `AndroidManifest.xml` (permissões, declaração de serviços) e os arquivos de configuração de segurança de rede discutidos no Capítulo 6. O Flutter gera essa pasta automaticamente, mas ela continua sendo editável como qualquer projeto Android nativo — é exatamente o que foi feito no Jarbas para declarar o serviço de primeiro plano do Modo Jarbas.

3.4 O ciclo de widgets raiz: `MaterialApp` e `Scaffold`

Quase todo app Flutter para Android/iOS começa envolvendo a árvore de widgets em um `MaterialApp` (ver [MaterialApp](#)), que fornece tema, rotas de navegação e widgets de acessibilidade padrão do Material Design do Google. Dentro dele, cada tela normalmente usa um `Scaffold`, que já resolve a estrutura visual comum de um app — barra superior (`AppBar`), corpo da tela, menus flutuantes etc. O Jarbas segue exatamente esse padrão, como se vê em `lib/main.dart` (Capítulo 10.2).

4. Introdução à linguagem Dart

Dart é a linguagem criada e mantida pelo Google especificamente para servir de base ao Flutter, embora seja de propósito geral (também roda em servidor). É uma linguagem com tipagem estática e forte, compilada — em desenvolvimento, para uma VM com JIT (permitindo o hot reload citado no Capítulo 2); em produção, para código de máquina nativo via AOT (Ahead-Of-Time), o que dá ao app final desempenho comparável a Kotlin/Swift nativos. A referência oficial completa é o [Dart Language Tour](#).

4.1 Conceitos essenciais usados no Jarbas

- **Null safety (segurança contra nulos).** Desde o Dart 2.12, o compilador distingue em tempo de compilação tipos que podem ser nulos (`String?`) dos que nunca podem (`String`), eliminando uma classe inteira de erros em tempo de execução. Ver [Sound null safety](#). No Jarbas isso aparece, por exemplo, em `SettingsStore.getBaseUrl()`, que retorna `Future<String?>` porque a URL pode simplesmente não ter sido configurada ainda.
- **Programação assíncrona com Future e async/await.** Toda operação que não termina instantaneamente — ler do disco, fazer uma requisição HTTP, esperar o microfone — retorna um `Future<T>`, e a sintaxe `async/await` permite escrever esse código de forma sequencial e legível em vez de encadear callbacks. Ver [Asynchronous programming: futures, async, await](#). Praticamente toda função do Jarbas que toca em rede, armazenamento ou microfone é `async`.
- **Streams.** Para eventos que acontecem repetidamente ao longo do tempo (cada trecho de áudio reconhecido pelo Vosk, por exemplo), o Dart usa `Stream<T>`, consumido com `.listen((valor) { ... })`. Ver [Streams](#). O Jarbas assina um stream de resultados do reconhecedor Vosk em `jarbas_service.dart` (Capítulo 10.6).
- **Isolates.** Dart não compartilha memória entre threads como Java/Kotlin; em vez disso, usa *isolates* — unidades de execução independentes que só se comunicam por troca de mensagens, evitando problemas clássicos de concorrência (race conditions). Ver [Concurrency in Dart](#). Esse conceito é central no Modo Jarbas: o serviço de escuta contínua roda em uma isolate separada da interface, sobrevivendo mesmo com a tela apagada (detalhado no Capítulo 5.3).
- **Classes, herança e tratamento de exceções tipado.** Dart é orientado a objetos com sintaxe próxima de Java/Kotlin/C#. O Jarbas usa uma hierarquia de exceções customizadas (`HaServiceException` e subclasses) para diferenciar, por tipo, falhas de rede, timeout e autenticação — ver [Exceptions](#).

4.2 Bons tutoriais em português

Para quem está começando em Dart/Flutter e prefere conteúdo em português, os recursos abaixo são gratuitos e bem estabelecidos na comunidade brasileira:

- **Flutterando** — comunidade brasileira dedicada exclusivamente a Flutter (podcast, YouTube e cursos), com uma trilha gratuita de [Lógica de Programação com Dart](#) voltada a quem nunca programou, além de turmas de [Flutter para Iniciantes](#).
- **DIO (Digital Innovation One)** — plataforma brasileira de ensino técnico com cursos gratuitos de [Fundamentos da linguagem Dart](#) e de [Flutter](#) cobrindo desde a base da linguagem até persistência de dados e gerência de estado.

- **Documentação oficial (em inglês, mas com exemplos executáveis)** — o [Dart Language Tour](#) e o [Flutter: Get started](#) continuam sendo a referência mais atualizada; mesmo para quem prefere estudar em português, vale usá-los como material de consulta pontual, já que tutoriais em vídeo ficam desatualizados mais rápido que a documentação oficial.

5. Por que o Jarbas é um bom estudo de caso de Flutter

Este relatório usa o Jarbas como estudo de caso porque ele toca, de forma concreta e testável em hardware real, nos três eixos mais comuns — e mais didáticos — de uma aplicação Flutter de produção: (1) uso extensivo dos widgets principais do framework, (2) comunicação em rede com uma API HTTP externa e (3) uso de recursos avançados de hardware do Android. As três seções a seguir detalham cada eixo com trechos reais do código.

5.1 Aplicação dos principais widgets do Flutter

A interface do Jarbas (arquivos `home_screen.dart` e `settings_screen.dart`) foi construída inteiramente com o catálogo padrão de widgets Material — ver o [catálogo de widgets do Flutter](#) — sem nenhuma biblioteca de UI de terceiros. Isso a torna um bom material de estudo porque cobre, num único app pequeno, os widgets mais usados no dia a dia:

- `Scaffold` e `AppBar` — esqueleto padrão de cada tela (`HomeScreen` e `SettingsScreen`).
- `Column`, `Row`, `Padding`, `SizeBox` e `Expanded` — os widgets de layout que organizam o espaço verticalmente/horizontalmente; usados para empilhar o indicador do Modo Jarbas, o botão de microfone e a grade de atalhos na tela principal.
- `GridView.builder` com `SliverGridDelegateWithFixedCrossAxisCount` — grade rolável de atalhos de comando (2 colunas), construída sob demanda (apenas os itens visíveis são materializados), evitando desperdício de memória quando o usuário cadastra muitos atalhos.
- `StatefulWidget/State` com `setState()` — todo o gerenciamento de estado da UI (estado do microfone, texto reconhecido, indicador do Modo Jarbas) usa o mecanismo mais básico e explícito do Flutter, sem biblioteca de gerência de estado externa (`Provider`, `Riverpod`, `Bloc`...) — uma escolha deliberada para manter o projeto pedagógico e sem dependências extras.
- `GestureDetector` e `InkWell/Material` — captura de toque no botão de microfone (`GestureDetector` sobre um `CircleAvatar`) e nos cartões de atalho (`InkWell` sobre `Material`, que dá o efeito visual de “ondulação” ao toque, padrão do Material Design).
- `TextField` e `TextEditingController` — campos de formulário da tela de configurações (URL, token, novo atalho).
- `ListTile`, `Switch` e `ListView` — indicador/controle do Modo Jarbas e lista de atalhos cadastrados.
- `Navigator/MaterialPageRoute` — navegação entre a tela principal e a de configurações.
- `ScaffoldMessenger/SnackBar` — feedback transitório (“Configurações salvas”).

A cobertura desses widgets em um único projeto pequeno permite usar o Jarbas como material de estudo completo do catálogo básico do Flutter sem precisar recorrer a múltiplos exemplos espalhados.

5.2 Uso de comunicação em rede

O segundo eixo é a integração com uma API HTTP externa real — o Home Assistant — via o pacote oficial <http>. O Jarbas ilustra o padrão mais comum de acesso a rede em apps móveis: requisições REST autenticadas por token, com tratamento explícito de timeout, erro de conexão e erro de autenticação — descrito em detalhe no Capítulo 7 e no código de `lib/ha_service.dart` (Capítulo 10.3).

5.3 Uso de recursos avançados de hardware

O terceiro e mais avançado eixo é o uso de hardware do aparelho além da tela e do toque: o **microfone**, acessado de duas formas complementares:

- **Reconhecimento de fala sob demanda**, via o pacote [speech_to_text](#), que envolve a API nativa do Android [SpeechRecognizer/RecognizerIntent](#) (tipicamente apoiada nos serviços de reconhecimento de fala do Google) — usada tanto no modo de toque no microfone ([home_screen.dart](#)) quanto para capturar o comando de voz depois que a wake word é detectada ([jarbas_service.dart](#)).
- **Detecção contínua e offline de palavra de ativação** (“OK Jarbas”), via o motor [Vosk](#) — um reconhecedor de fala totalmente local, sem depender de rede nem de conta em nuvem, escolhido em substituição ao Picovoice/Porcupine (justificativa completa no Capítulo 9). Esse motor precisa rodar de forma persistente mesmo com a tela apagada, o que exige um **serviço em primeiro plano** ([foreground service](#), via o pacote [flutter_foreground_task](#)) rodando em uma isolate Dart separada da interface — um exemplo prático do conceito de isolates do Capítulo 4.1.

Complementando a experiência, o app também usa [vibration](#) (retorno tátil ao detectar a wake word) e [audioplayers](#) (retorno sonoro, um beep local, sem áudio externo). Ambos os casos exigiram uma lição de arquitetura registrada no Capítulo 10.6: plugins que dependem de bindings do Flutter só podem ser inicializados depois que o ciclo de vida da tarefa em segundo plano começa de fato — nunca antes, mesmo que pareça inofensivo.

6. Configurações de rede e segurança no Android

Todo acesso a rede e a hardware sensível (microfone) em Android exige declaração explícita de permissões e, quando aplicável, de política de segurança de rede. O Jarbas concentra essas declarações em dois arquivos dentro de `android/app/src/main/`.

6.1 Permissões no AndroidManifest.xml

O Android usa um modelo de [permissões declaradas em manifesto](#): toda capacidade sensível (rede, microfone, notificações, execução em primeiro plano) precisa ser listada com uma tag `<uses-permission>`. O Jarbas declara:

android/app/src/main/AndroidManifest.xml (trecho de permissões).

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_MICROPHONE"/>
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

- **INTERNET** — necessária para toda chamada HTTP ao Home Assistant.
- **RECORD_AUDIO** — necessária tanto para o `speech_to_text` quanto para o Vosk capturarem áudio do microfone.
- **FOREGROUND_SERVICE** e **FOREGROUND_SERVICE_MICROPHONE** — desde o Android 14 (API 34), todo serviço em primeiro plano precisa declarar seu *tipo*; como o Modo Jarbas usa o microfone continuamente, o tipo declarado é `microphone` (ver [Foreground service types](#)). Sem essas permissões, o sistema recusa o `startForegroundService` em tempo de execução.
- **POST_NOTIFICATIONS** — desde o Android 13 (API 33), mostrar qualquer notificação (incluindo a notificação persistente obrigatória de um foreground service) exige essa permissão de execução, solicitada explicitamente ao usuário (ver `FlutterForegroundTask.requestNotificationPermission()` em `home_screen.dart`).
- **BLUETOOTH_CONNECT** — dependência transitiva de plugins do projeto (nenhuma funcionalidade Bluetooth é usada diretamente pelo Jarbas hoje).

Note que declarar a permissão no manifesto é necessário mas não suficiente: **RECORD_AUDIO** e **POST_NOTIFICATIONS** são “perigosas”/de execução ([runtime permissions](#)) e também precisam ser concedidas pelo usuário em tempo de execução — papel cumprido pelo pacote [permission_handler](#) e pela própria API do `speech_to_text/flutter_foreground_task`.

6.2 Network Security Configuration e tráfego HTTP sem TLS

Desde o Android 9 (API 28), aplicativos não podem fazer requisições HTTP em texto puro (sem TLS) por padrão — apenas HTTPS (ver [Network security configuration](#)). O Home Assistant do usuário, no entanto, é acessado majoritariamente por **IP local dinâmico** (DHCP), sem certificado TLS válido — um cenário comum em automação residencial doméstica, onde a instância do HA normalmente não tem um domínio público nem certificado emitido por uma autoridade confiável.

A solução aplicada foi declarar explicitamente essa exceção em um arquivo de configuração de segurança de rede dedicado:

android/app/src/main/res/xml/network_security_config.xml

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <base-config cleartextTrafficPermitted="true" />
</network-security-config>
```

referenciado no manifesto via `android:networkSecurityConfig="@xml/network_security_config"`. Uma alternativa mais restritiva seria usar `<domain-config>` para liberar cleartext apenas para domínios específicos — mas o Android não aceita faixas de IP (CIDR) nem hosts dinâmicos nesse elemento, e o IP do Home Assistant muda por DHCP. A alternativa escolhida (`base-config` global) é documentada e compreendida como uma redução deliberada de segurança de transporte, aceitável neste contexto porque o tráfego roda inteiramente dentro da rede local confiável do usuário (e, quando fora de casa, passa por uma VPN WireGuard via [Tailscale](#), que já criptografa o túnel independentemente do HTTP interno) — ver a referência OWASP sobre o tema em [MASTG-KNOW-0014: Android Network Security Configuration](#). Em um cenário de exposição direta à internet, o recomendado seria expor o Home Assistant atrás de HTTPS (o próprio HA suporta isso nativamente) e reverter essa configuração.

7. HTTP, APIs REST e autenticação por Bearer Token

O Jarbas não implementa nenhuma lógica própria de interpretação de comandos de voz — ele delega 100% dessa responsabilidade ao motor de linguagem natural do próprio Home Assistant (o “Assist”), comunicando-se com ele por uma API HTTP REST simples. Essa escolha de arquitetura foi deliberada e traz vantagens concretas que justificam o padrão adotado.

7.1 Por que REST sobre HTTP

- **Simplicidade e portabilidade.** HTTP é o protocolo mais universalmente suportado — qualquer linguagem/SDK sabe fazer um [GET/POST](#). Não é necessário abrir/manter conexões persistentes (como WebSocket) para o caso de uso do Jarbas, em que cada comando de voz é uma interação isolada e de curta duração.
- **Sem estado no cliente.** Cada requisição carrega tudo que o servidor precisa (texto do comando + token de autorização), seguindo o princípio REST de “stateless” — o app não precisa manter sessão nem lidar com reconexão após o aparelho ficar sem rede momentaneamente.
- **API já existente e estável no Home Assistant.** O endpoint [/api/conversation/process](#) (ver [Home Assistant Conversation API](#) e a integração [Conversation](#)) já existe, é mantido pelo próprio projeto HA e evolui com ele — o Jarbas só precisa consumir, nunca reimplementar, o entendimento de linguagem natural.

7.2 O contrato HTTP usado pelo Jarbas

O arquivo [lib/ha_service.dart](#) concentra toda a comunicação de rede em uma única classe, [HaService](#), com dois métodos:

lib/ha_service.dart — requisição autenticada ao Assist do Home Assistant.

```
Future<http.Response> _postConversation(  
  String baseUrl,  
  String token,  
  String text,  
) async {  
  return await _client  
    .post(  
      Uri.parse('$baseUrl/api/conversation/process'),  
      headers: {  
        'Authorization': 'Bearer $token',  
        'Content-Type': 'application/json',  
      },  
      body: jsonEncode({'text': text, 'language': 'pt-BR'}),  
    )  
    .timeout(_timeout);  
}
```

O cabeçalho [Authorization: Bearer <token>](#) segue o esquema “Bearer” definido pela [RFC 6750 — OAuth 2.0 Bearer Token Usage](#): qualquer parte que possua o token pode usá-lo para acessar o recurso protegido, sem precisar provar posse de uma chave criptográfica adicional a cada chamada — daí o nome “bearer” (“portador”). É o esquema de autenticação mais simples e amplamente suportado para acesso a APIs REST (ver também [Bearer Tokens — OAuth 2.0](#)).

O Home Assistant simplifica ainda mais esse fluxo para uso pessoal: em vez de um fluxo OAuth completo (redirecionamento, troca de código por token, renovação por refresh token), o usuário gera um [Long-Lived Access Token](#) diretamente no seu perfil — um token de longa duração, revogável manualmente, pensado exatamente para scripts e integrações de uso pessoal como o Jarbas (ver

também a [Authentication API](#) do HA para o fluxo OAuth completo, usado por integrações de terceiros publicadas). No app, esse token é apenas um campo de configuração inserido pelo usuário ([SettingsScreen](#)) e persistido localmente — nunca embutido (“hardcoded”) no código-fonte.

7.3 Tratamento defensivo de erros de rede

Toda chamada de rede que atualiza a interface precisa lidar com pelo menos três categorias de falha — tempo esgotado, falha de conexão e credencial inválida — sem travar a UI. O Jarbas tipifica essas falhas como exceções próprias:

lib/ha_service.dart — hierarquia de exceções tipadas.

```
class HaUnauthorizedException extends HaServiceException {
  const HaUnauthorizedException() : super('Token inválido ou não autorizado.');
```

```
}

class HaTimeoutException extends HaServiceException {
  const HaTimeoutException() : super('Tempo de resposta esgotado.');
```

```
}

class HaConnectionException extends HaServiceException {
  const HaConnectionException()
    : super('Falha ao conectar ao Home Assistant.');
```

```
}
```

Essa tipagem específica é combinada com uma rede de segurança genérica (`catch (_)`) nos pontos de entrada de rede — uma lição de arquitetura obtida na prática: um `catch` específico demais em `testConnection()` deixava passar um `ArgumentError` de URL vazia/mal formada e travava o indicador de carregamento para sempre, sem mensagem de erro visível ao usuário. Qualquer chamada de rede que altera estado de UI precisa de um `catch` amplo como última linha de defesa, além dos tipos de exceção específicos esperados (detalhes no Capítulo 10.3).

8. Home Assistant: contexto para leigos

[Home Assistant](#) é uma plataforma de automação residencial de código aberto, autogerenciada (o usuário roda o servidor na própria rede, tipicamente em um Raspberry Pi, mini-PC ou máquina virtual) — diferente de soluções fechadas como Google Home ou Amazon Alexa, em que a inteligência central roda na nuvem do fabricante. Todo o processamento e armazenamento de dados do Home Assistant pode ficar dentro de casa, sem depender de internet para as automações funcionarem.

Para quem nunca usou: o Home Assistant se conecta a milhares de fabricantes de dispositivos (lâmpadas inteligentes, tomadas, fechaduras, sensores de temperatura, robôs aspiradores, etc.) por meio de “integrações”, e representa cada capacidade desses dispositivos como uma **entidade** — por exemplo, `light.spot_mesa` (uma luz) ou `lock.porta_frente` (uma fechadura). Cada entidade tem um **estado** (“ligado”, “desligado”, “22.5°C”) que pode ser lido e, quando aplicável, alterado — acender uma luz é, tecnicamente, mudar o estado de uma entidade de `off` para `on`.

A peça relevante para o Jarbas é o [Assist](#), o assistente de voz/texto embutido no próprio Home Assistant: ele recebe uma frase em linguagem natural (“acender as luzes da sala”), interpreta a intenção, localiza a(s) entidade(s) correspondente(s) pelo nome (ou por um “alias” cadastrado) e executa a ação — tudo isso *dentro do próprio HA*. O Jarbas não precisa (nem tenta) entender comandos por conta própria: ele apenas transcreve a fala em texto (usando o motor de reconhecimento do Android ou o Vosk, conforme o modo) e repassa esse texto ao endpoint `/api/conversation/process` do Assist, devolvendo ao usuário a resposta falada/textual que o próprio HA gera. Essa divisão de responsabilidade é o que permite ao Jarbas ser um cliente de voz simples: toda a complexidade de interpretar “desligar tudo” ou “abaixar o volume da sala” já é resolvida, mantida e evoluída pelo próprio projeto Home Assistant.

9. A escolha do Vosk em detrimento do Picovoice

Uma “wake word” (palavra de ativação) é uma frase curta reconhecida continuamente por um modelo pequeno e de baixíssimo consumo de energia, rodando localmente 24 horas por dia, cujo único trabalho é decidir “alguém acabou de dizer essa palavra específica?” — é o mecanismo por trás de “OK Google”, “Alexa” ou, no caso deste projeto, “OK Jarbas”. Só depois da detecção é que um reconhecedor de fala completo (mais pesado) é acionado para capturar o comando real.

9.1 Tentativa inicial: Picovoice / Porcupine

A primeira escolha de motor de wake word foi o [Porcupine](#), produto da [Picovoice](#), por ser leve, preciso e ter pacote Flutter oficial. O fluxo de uso normal do Porcupine exige criar uma conta gratuita no *Picovoice Console* e gerar uma *AccessKey* — sem custo para uso pessoal em pequena escala (o serviço oferece um nível gratuito para até três usuários ativos por mês, com planos comerciais pagos a partir daí).

Na prática, porém, a conta criada para este projeto — tentada com três e-mails diferentes (corporativo, pessoal e educacional) — ficou presa indefinidamente em “revisão manual de caso de uso comercial”, sem liberação de *AccessKey* e sem prazo estimado de resposta. Esse gargalo operacional (dependência de aprovação humana de terceiro, fora do controle do desenvolvedor, para uma funcionalidade central do app) foi o motivo decisivo para abandonar o Picovoice, independentemente da qualidade técnica do produto em si.

9.2 Escolha final: Vosk

[Vosk](#) é um kit de reconhecimento de fala **totalmente offline e de código aberto**, mantido pela AlphaCephei (ver o repositório [alphacep/vosk-api](#)), com modelos pequenos (na ordem de dezenas de MB) para mais de 20 idiomas — incluindo português — disponíveis diretamente em [alphacephei.com/vosk/models](#). Diferente do Porcupine, o Vosk não é um motor especializado só em detectar uma palavra-chave: é um reconhecedor de fala contínuo de vocabulário geral, que o Jarbas restringe artificialmente a duas saídas possíveis usando uma *gramática* — `['ok jarbas', '[unk]']` — fazendo-o funcionar, na prática, como um detector de wake word de baixo custo computacional.

As vantagens que motivaram a migração foram, em ordem de importância para este projeto: (1) não exige conta, chave de API nem aprovação de terceiros — o modelo é baixado uma vez e roda inteiramente no aparelho; (2) licença livre (Apache 2.0), compatível com um projeto sem intenção comercial; (3) suporte nativo a português sem custo adicional.

9.3 Comparativo

Critério	Picovoice / Porcupine	Vosk
Custo/licenciamento	Gratuito até 3 usuários/mês; planos comerciais a partir de US\$ 899/mês	Código aberto (Apache 2.0), sem custo
Necessidade de conta	Sim — AccessKey via Picovoice Console, sujeito a revisão manual	Não — modelo baixado localmente
Funcionamento offline	Sim	Sim
Especialização	Motor dedicado a wake word (altíssima eficiência energética)	Reconhecedor de fala geral, restringido por gramática
Tamanho do modelo	Poucos KB por palavra-chave	~31 MB (modelo pequeno pt-BR usado no projeto)
Pacote Flutter usado	porcupine_flutter (oficial, abandonado neste projeto)	vosk_flutter_service (fork ativo — ver Capítulo 9.4)

9.4 Nota técnica: escolha do pacote Flutter para o Vosk

Mesmo após decidir pelo motor, restava escolher o pacote Flutter que o expõe em Dart. Três candidatos foram avaliados e descartados antes da escolha final:

- `vosk_flutter_2` — exige `minSdkVersion 30`; o Note 9 dedicado ao Modo Jarbas roda Android 10 (API 29) e nem instalaria o app.
- `vosk_flutter` (pacote oficial) — abandonado desde 2023, com restrição de SDK Dart incompatível com a versão usada no projeto.
- `vosk_flutter_fixed` — build quebrava por inconsistência entre o `compileSdk` declarado pelo pacote e o exigido por suas próprias dependências transitivas (mesma classe de problema que já havia ocorrido com o Porcupine, descrita no Capítulo 10.9).

O pacote adotado, `vosk_flutter_service`, resolveu as três restrições simultaneamente (`minSdkVersion 21`, compatível com Dart 3, `compileSdk` consistente com suas dependências). O efeito colateral foi elevar o `compileSdk` do próprio projeto Android para 37, documentado no Capítulo 10.9. A lição geral registrada no projeto: ao escolher um pacote Flutter com binário nativo Android, vale sempre checar três coisas *antes* de codar — o `minSdkVersion` do pacote (compatibilidade com o aparelho alvo), o `environment.sdk` do seu `pubspec.yaml` (compatibilidade com a versão de Dart/Flutter do projeto) e se o `compileSdk` do próprio pacote é consistente com o exigido por suas dependências transitivas.

10. Anatomia detalhada do código

Este capítulo percorre, arquivo por arquivo, todo o código-fonte e toda a configuração da aplicação Jarbas — a versão em produção no repositório [Beto-Inovacao-LTDA/jarbas](https://github.com/Beto-Inovacao-LTDA/jarbas). Serve como referência para quem for manter, revisar ou estender o projeto.

10.1 pubspec.yaml — manifesto do pacote

Declara nome do pacote, SDK Dart mínimo, todas as dependências de terceiros e quais arquivos de `assets/` são empacotados no aplicativo final.

pubspec.yaml (comentários originais omitidos por brevidade).

```
name: ha_voice_app
version: 1.0.0+1

environment:
  sdk: ^3.13.3

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  speech_to_text: ^7.0.0
  http: ^1.2.2
  shared_preferences: ^2.3.3
  permission_handler: ^13.0.0
  vosk_flutter_service: ^0.1.3
  flutter_foreground_task: ^8.7.0
  vibration: ^3.2.1
  audioplayers: ^6.8.1

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
  flutter_launcher_icons: ^0.14.3

flutter:
  uses-material-design: true
  assets:
    - assets/vosk/vosk-model-small-pt-0.3.zip
    - assets/sounds/wake_beep.wav
```

Cada dependência corresponde a uma responsabilidade específica já discutida: `http` (Capítulo 7), `shared_preferences` (persistência local de configurações — Capítulo 10.5), `speech_to_text` e `vosk_flutter_service` (reconhecimento de voz sob demanda e wake word offline — Capítulos 5.3 e 9), `flutter_foreground_task` (serviço em primeiro plano do Modo Jarbas), `permission_handler` (permissões de execução) e `vibration/audioplayers` (feedback tátil/sonoro). O modelo de linguagem do Vosk (~31 MB) é embarcado como asset binário do próprio app, permitindo funcionamento 100% offline desde a primeira execução.

10.2 lib/main.dart — ponto de entrada

lib/main.dart (30 linhas).

```

import 'package:flutter/material.dart';
import 'package:flutter_foreground_task/flutter_foreground_task.dart';

import 'home_screen.dart';

void main() {
  FlutterForegroundTask.initCommunicationPort();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final colorScheme = ColorScheme.fromSeed(seedColor: Colors.deepPurple);
    return MaterialApp(
      title: 'Jarbas',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: colorScheme,
        appBarTheme: AppBarTheme(
          backgroundColor: colorScheme.primary,
          foregroundColor: Colors.white,
        ),
      ),
      home: const HomeScreen(),
    );
  }
}

```

O menor e mais simples arquivo do projeto, mas com um detalhe não óbvio: `FlutterForegroundTask.initCommunicationPort()` precisa ser chamado antes de `runApp()` para abrir o canal de mensagens entre a isolate principal (UI) e a isolate do serviço em segundo plano do Modo Jarbas (Capítulo 10.6) — sem essa linha, `FlutterForegroundTask.addTaskDataCallback` em `home_screen.dart` nunca recebe atualizações de status. `MyApp` é um `StatelessWidget` (Capítulo 3.2) que só define tema (esquema de cores gerado a partir de um roxo semente, `ColorScheme.fromSeed`) e a tela inicial (`HomeScreen`).

10.3 lib/ha_service.dart — cliente HTTP do Home Assistant

lib/ha_service.dart (102 linhas — trecho com a lógica principal).

```

class HaService {
  HaService({http.Client? client, Duration? timeout})
    : _client = client ?? http.Client(),
      _timeout = timeout ?? const Duration(seconds: 10);

  final http.Client _client;
  final Duration _timeout;

  Future<String> sendCommand(String baseUrl, String token, String text) async {
    final response = await _postConversation(baseUrl, token, text);
    _checkStatus(response);
    try {
      final body = jsonDecode(response.body) as Map<String, dynamic>;
      final speech = body['response']['speech']['plain']['speech'] as String;
      return speech;
    } catch (_) {
      throw const HaUnexpectedResponseException();
    }
  }

  Future<bool> testConnection(String baseUrl, String token) async {
    try {
      final response = await _client
        .get(Uri.parse('$baseUrl/api/'),
            headers: {'Authorization': 'Bearer $token'})
        .timeout(_timeout);
      return response.statusCode >= 200 && response.statusCode < 300;
    } catch (_) {
      return false;
    }
  }

  Future<http.Response> _postConversation(
    String baseUrl, String token, String text) async {
    try {
      return await _client
        .post(Uri.parse('$baseUrl/api/conversation/process'),
            headers: {
              'Authorization': 'Bearer $token',
              'Content-Type': 'application/json',
            },
            body: jsonEncode({'text': text, 'language': 'pt-BR'}))
        .timeout(_timeout);
    } on TimeoutException {
      throw const HaTimeoutException();
    } catch (_) {
      throw const HaConnectionException();
    }
  }

  void _checkStatus(http.Response response) {
    if (response.statusCode == 401) throw const HaUnauthorizedException();
    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw const HaUnexpectedResponseException();
    }
  }
}

```

Único ponto de contato HTTP de todo o app com o mundo externo — usado tanto pela tela principal (modo sob demanda) quanto pelo serviço de segundo plano (Modo Jarbas), cada um instanciando sua própria cópia de `HaService` (isolates não compartilham objetos, ver Capítulo 4.1). `sendCommand()` faz o POST autenticado e extrai o campo de texto falado da resposta JSON do Assist (`response.speech.plain.speech`); `testConnection()` apenas confirma que a URL e o token funcionam, usada pelo botão “Testar conexão” da tela de configurações. Os detalhes de protocolo e tratamento de erro estão no Capítulo 7.

10.4 lib/home_screen.dart — tela principal

A tela mais extensa do app (334 linhas), responsável por três blocos de interface: o indicador/controle do Modo Jarbas, o botão de microfone do modo sob demanda, e a grade de atalhos de comando.

lib/home_screen.dart (trecho representativo).

```
enum _MicState { idle, listening, sending }

class _HomeScreenState extends State<HomeScreen> {
  late final SettingsStore _store = widget.store ?? SettingsStore();
  late final HaService _haService = widget.haService ?? HaService();
  late final stt.SpeechToText _speech = widget.speech ?? stt.SpeechToText();

  List<Shortcut> _shortcuts = [];
  _MicState _micState = _MicState.idle;
  bool _jarbasActive = false;

  @override
  void initState() {
    super.initState();
    _loadShortcuts();
    FlutterForegroundTask.addTaskDataCallback(_onJarbasData);
    _syncJarbasState();
  }

  Future<void> _toggleListening() async {
    if (_micState == _MicState.listening) {
      await _speech.stop();
      setState(() => _micState = _MicState.idle);
      return;
    }
    final available = await _ensureSpeechInitialized();
    if (!available) return;
    setState(() => _micState = _MicState.listening);
    await _speech.listen(
      listenOptions: stt.SpeechListenOptions(localeId: 'pt_BR'),
      onResult: (result) {
        if (result.finalResult && result.recognizedWords.isNotEmpty) {
          _sendCommand(result.recognizedWords);
        }
      },
    );
  }

  void _onJarbasData(Object data) {
    if (data is! Map) return;
    final status = data['status'] as String?;
    setState(() {
      if (status == 'error') {
        _jarbasActive = false;
      } else if (status != null) {
        _jarbasActive = true;
      }
    });
  }

  // ... build() monta Scaffold > Column > [ListTile+Switch, mic, GridView.builder]
}
```

Três pontos merecem destaque. Primeiro, a inicialização preguiçosa do `speech_to_text`: `_ensureSpeechInitialized()` só é chamado no primeiro toque do microfone, nunca em `initState()` — isso evita solicitar permissão de microfone antes da hora e evita que testes de widget que não usam o microfone toquem o plugin real. Segundo, a comunicação entre isolates: `FlutterForegroundTask.addTaskDataCallback(_onJarbasData)` registra um ouvinte para mensagens vindas da isolate do Modo Jarbas (Capítulo 10.6), atualizando o indicador visual em tempo real sem polling. Terceiro, a grade de atalhos usa `GridView.builder` com `SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 2, mainAxisExtent: 56)`, construindo apenas os cartões visíveis (Capítulo 5.1).

10.5 lib/settings_store.dart — persistência local

lib/settings_store.dart (80 linhas — trecho representativo).

```
class SettingsStore {
  static const _keyBaseUrl = 'ha_base_url';
  static const _keyToken = 'ha_token';
  static const _keyShortcuts = 'ha_shortcuts';

  Future<String?> getBaseUrl() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_keyBaseUrl);
  }

  Future<void> setBaseUrl(String value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_keyBaseUrl, value);
  }

  Future<List<Shortcut>> getShortcuts() async {
    final prefs = await SharedPreferences.getInstance();
    final raw = prefs.getString(_keyShortcuts);
    if (raw == null) {
      final defaults = defaultShortcuts();
      await setShortcuts(defaults);
      return defaults;
    }
    final list = jsonDecode(raw) as List<dynamic>;
    return list.map((item) => Shortcut.fromJson(item)).toList();
  }
  // setShortcuts(), getToken()/setToken(), getJarbasAutostart()/setJarbasAutostart()
  // seguem o mesmo padrão de leitura/escrita chave-valor.
}
```

Encapsula o pacote [shared_preferences](#), que grava pares chave-valor simples em armazenamento nativo persistente do Android/iOS (XML no Android, plist no iOS). Guarda a URL e o token do Home Assistant, a lista de atalhos de comando (serializada como JSON dentro de uma única chave string) e a preferência de início automático do Modo Jarbas. A classe [Shortcut](#) (par de [label/phrase](#)) implementa [toJson\(\)/fromJson\(\)](#) manualmente, sem geração de código, adequado ao tamanho pequeno do modelo de dados do app.

10.6 lib/jarbas_service.dart — Modo Jarbas (wake word e serviço)

O arquivo mais complexo do projeto (300 linhas): implementa a detecção contínua da wake word via Vosk, a captura do comando de voz seguinte via [speech_to_text](#), e a orquestração de tudo dentro de um serviço em primeiro plano com sua própria isolate.

lib/jarbas_service.dart (trecho consolidado das partes centrais).

```

class JarbasService {
  static Future<ServiceRequestResult> start() async {
    _ensureInitialized();
    if (await FlutterForegroundTask.isRunningService) {
      return const ServiceRequestSuccess();
    }
    return FlutterForegroundTask.startService(
      serviceId: _serviceId,
      notificationTitle: 'Jarbas',
      notificationText: 'Ouvindo "OK Jarbas"...',
      callback: startJarbasTaskCallback,
    );
  }
}

@pragma('vm:entry-point')
void startJarbasTaskCallback() {
  FlutterForegroundTask.setTaskHandler(JarbasTaskHandler());
}

class JarbasTaskHandler extends TaskHandler {
  AudioPlayer? _beepPlayer;

  @override
  Future<void> onStart(DateTime timestamp, TaskStarter starter) async {
    // Criado aqui, não como campo de classe: o construtor de AudioPlayer
    // acessa bindings do Flutter que só existem depois que a isolate de
    // segundo plano termina de inicializar.
    _beepPlayer = AudioPlayer();
    final modelPath = await ModelLoader().loadFromAssets(_modelAssetPath);
    final model = await VoskFlutterPlugin.instance().createModel(modelPath);
    _voskRecognizer = await VoskFlutterPlugin.instance().createRecognizer(
      model: model, sampleRate: 16000, grammar: ['ok jarbas', '[unk]'],
    );
    await _startWakeWordListening();
  }

  Future<void> _startWakeWordListening() async {
    final speechService =
      await VoskFlutterPlugin.instance().initSpeechService(_voskRecognizer!);
    _voskSpeechService = speechService;
    _voskResultSubscription = speechService.onResult().listen(_onVoskResult);
    await speechService.start(onRecognitionError: _onVoskRecognitionError);
  }

  Future<void> _stopWakeWordListening() async {
    await _voskResultSubscription?.cancel();
    await _voskSpeechService?.stop(); // stop() ANTES de dispose(): ver texto
    await _voskSpeechService?.dispose();
  }

  void _onVoskResult(String resultJson) {
    final text = _extractRecognizedText(resultJson);
    if (text != 'ok jarbas') return;
    unawaited(_onWakeWordDetected());
  }

  Future<void> _onWakeWordDetected() async {
    await _stopWakeWordListening();
    unawaited(_emitWakeWordFeedback()); // vibração + beep
    final text = await _captureCommand(); // speech_to_text
    final speech = await _haService!.sendCommand(baseUrl, token, text);
    await _startWakeWordListening(); // volta a escutar
  }
}

```

Este arquivo concentra as três lições de engenharia mais importantes do projeto, cada uma decorrente de um crash real reproduzido no aparelho físico (Galaxy Note 9, Android 10):

- **Ordem obrigatória `stop()` antes de `dispose()`** — o `SpeechService` nativo do Vosk roda uma thread Java crua que lê continuamente do `AudioRecord`; se `dispose()` for chamado enquanto essa thread ainda está lendo, ela lança `RuntimeException("error reading audio buffer")` não capturável do lado Dart, derrubando o processo inteiro. Esse foi exatamente o crash observado ao desativar o Modo Jarbas antes da correção.
- **Nunca reiniciar o mesmo `AudioRecord` depois de outro app usar o microfone** — depois que o `speech_to_text` (Google) captura o comando de voz, o `JarbasTaskHandler` descarta completamente o `SpeechService` anterior e cria um novo em cima do mesmo `Recognizer/Model` já carregados (sem recarregar os assets) — reiniciar o `AudioRecord` antigo, em vez de recriar, falhava e derrubava o app silenciosamente.
- **Plugins dependentes de bindings do Flutter só podem ser inicializados dentro de `onStart()`**, nunca como inicializador de campo de classe — um `AudioPlayer()` criado como campo lançava “Binding has not yet been initialized”, engolido silenciosamente por um `try/catch` de melhor esforço, de forma que o beep sonoro simplesmente nunca tocava (só a vibração funcionava). O mesmo motivo explica por que `HapticFeedback/SystemSound` padrão do Flutter não funcionam nessa isolate: dependem de `SystemChannels.platform`, que por sua vez exige uma `Activity` — inexistente numa isolate de segundo plano headless. Por isso o feedback tátil usa o pacote [vibration](#), que aciona o `Vibrator` do Android diretamente via `Context`.

10.7 lib/settings_screen.dart — tela de configurações

lib/settings_screen.dart (201 linhas — trecho representativo).

```
Future<void> _testConnection() async {
  setState(() => _testingConnection = true);
  final ok = await _haService.testConnection(
    _urlController.text, _tokenController.text);
  setState(() {
    _testingConnection = false;
    _testResultMessage = ok ? 'Conexão bem-sucedida' : 'Falha na conexão';
  });
}

Future<void> _addShortcut() async {
  final label = _shortcutLabelController.text.trim();
  final phrase = _shortcutPhraseController.text.trim();
  if (label.isEmpty || phrase.isEmpty) return;
  final updated = [..._shortcuts, Shortcut(label: label, phrase: phrase)];
  await _store.setShortcuts(updated);
  setState(() => _shortcuts = updated);
}
```

Tela convencional de formulário: campos de URL e token (`TextField` com `TextEditingController`), botão de teste de conexão que chama `HaService.testConnection()`, botão salvar que persiste via `SettingsStore`, e uma lista com CRUD simples de atalhos (`ListTile` + botão de remoção + dois campos para adicionar um novo). Todos os widgets recebem uma `Key` explícita (ex.: `Key('save_button')`) — não para a lógica de negócio, mas para permitir que os testes de widget do projeto (`test/settings_screen_test.dart`) localizem e interajam com cada elemento de forma estável.

10.8 android/app/src/main/AndroidManifest.xml

Já reproduzido e detalhado no Capítulo 6.1 (permissões) — o manifesto também declara a `MainActivity` (tema, orientação, modo de entrada de teclado), o serviço nativo do `flutter_foreground_task`

(`com.pravera.flutter_foreground_task.service.ForegroundService`, com `foregroundServiceType="microphone"`) e a referência ao arquivo de configuração de segurança de rede via `android:networkSecurityConfig="@xml/network_security_config"`.

10.9 android/app/build.gradle.kts

android/app/build.gradle.kts (trecho).

```
android {
    namespace = "com.betoinovacao.ha_voice_app"
    // permission_handler_android (dependência do vosk_flutter_service) exige
    // compileSdk 37; o default do Flutter nesta versão ainda é 36.
    compileSdk = 37
    ndkVersion = flutter.ndkVersion

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    defaultConfig {
        applicationId = "com.betoinovacao.ha_voice_app"
        minSdk = flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }
}
```

O `compileSdk = 37` explícito é a marca de um problema recorrente em projetos Flutter com plugins nativos: cada pacote publicado declara sua própria exigência mínima de SDK de compilação Android, e se uma dependência transitiva (aqui, `permission_handler_android`, puxada indiretamente pelo `vosk_flutter_service`) pedir uma versão maior que o padrão do projeto, o build falha com um erro de verificação de metadados do AAR (“AAR metadata check”) até que o `compileSdk` seja elevado manualmente — exatamente a mesma classe de incompatibilidade que já havia ocorrido antes com o `porcupine_flutter/flutter_voice_processor` (Picovoice), resolvida então por um patch de `compileSdkVersion` reescrito diretamente no cache de pacotes do Gradle. O `proguard-rules.pro` (Capítulo 10.11) trata de um risco correlato: o `vosk_flutter_service` usa JNA (Java Native Access) para acessar a biblioteca nativa do Vosk, e o R8/ProGuard poderia removê-la por engano num build com *shrinking* habilitado.

10.10 android/app/src/main/res/xml/network_security_config.xml

Já reproduzido e detalhado no Capítulo 6.2 — libera tráfego HTTP em texto puro (`cleartextTrafficPermitted="true"`) para toda a aplicação, necessário porque o Home Assistant é acessado por IP local dinâmico sem certificado TLS.

10.11 android/app/proguard-rules.pro

android/app/proguard-rules.pro (arquivo completo).

```
# vosk_flutter_service usa JNA para acessar a lib nativa do Vosk; exigido pelo
# README do pacote caso o shrinking (R8) seja habilitado no futuro (hoje
# minifyEnabled não está ativo neste projeto).
-keep class com.sun.jna.* { *; }
-keepclassmembers class * extends com.sun.jna.* { public *; }
```

Regras de preservação para o R8 (o compactador/ofuscador de código usado em builds de release do Android — ver [Shrink, obfuscate, and optimize your app](#)). Como o build de release deste projeto ainda

assina com a chave de debug e não tem `minifyEnabled` ativo (ver Capítulo 10.9), essas regras hoje não têm efeito prático — mas ficam registradas preventivamente, já que código acessado via reflexão/JNA é um alvo comum de remoção acidental por ofuscadores, e o próprio pacote recomenda essa precaução.

11. Conclusão

O Jarbas demonstra, em um projeto de escopo pequeno o suficiente para ser totalmente compreendido em um único relatório, o ciclo completo de desenvolvimento de um app Flutter de produção com integrações reais: consumo de uma API REST externa autenticada por token, dois modos distintos de acesso ao microfone (sob demanda e contínuo/offline), execução em segundo plano via serviço em primeiro plano com isolate dedicada, e as armadilhas de compatibilidade nativa (versões de SDK Android, ciclo de vida de plugins) que surgem inevitavelmente ao sair do conjunto de widgets puramente visuais e tocar hardware de verdade.

A decisão mais estrategicamente relevante documentada aqui — abandonar o Picovoice em favor do Vosk — não foi motivada por limitação técnica do produto descartado, mas por um risco de dependência de terceiros fora do controle do time (aprovação manual de conta). É uma lição que extrapola este projeto: ao escolher um serviço externo para uma funcionalidade central de um produto, o custo de troca (“switching cost”) e o grau de controle sobre o próprio roadmap pesam tanto quanto a qualidade técnica isolada da solução.

Os próximos passos naturais do projeto — não cobertos por este relatório por ainda estarem pendentes de execução — são a validação de consumo de bateria do Modo Jarbas em uso prolongado e a spec de testes/distribuição, registradas no diretório [specs/](#) do repositório.

12. Referências

Flutter e Dart

- [Flutter — site oficial](#)
- [Flutter architectural overview](#)
- [Flutter — Introduction to widgets](#)
- [Flutter — Widget catalog](#)
- [Flutter — Get started](#)
- [Flutter — Hot reload](#)
- [Flutter — Build and release an iOS app](#)
- [MaterialApp \(API reference\)](#)
- [pub.dev — repositório de pacotes Dart/Flutter](#)
- [Dart — Language tour](#)
- [Dart — Sound null safety](#)
- [Dart — Asynchronous programming \(futures, async, await\)](#)
- [Dart — Streams](#)
- [Dart — Concurrency in Dart \(isolates\)](#)
- [Dart — Error handling / exceptions](#)

Tutoriais em português

- [Flutterando — Lógica de Programação com Dart](#)
- [Flutterando — Flutter Iniciante](#)
- [DIO — Fundamentos da linguagem Dart](#)
- [DIO — Curso de Flutter](#)

Pacotes usados pelo Jarbas (pub.dev)

- [http](#)
- [shared_preferences](#)
- [speech_to_text](#)
- [permission_handler](#)
- [flutter_foreground_task](#)
- [vibration](#)
- [audioplayers](#)

Android — plataforma, rede e segurança

- [Android — Permissions overview](#)
- [Android — Requesting runtime permissions](#)
- [Android — Foreground service types](#)
- [Android — Network security configuration](#)
- [Android — SpeechRecognizer \(API reference\)](#)
- [Android — RecognizerIntent \(API reference\)](#)

- [Android — Shrink, obfuscate, and optimize your app \(R8/ProGuard\)](#)
- [OWASP MASTG — Android Network Security Configuration](#)

HTTP, REST e autenticação

- [RFC 6750 — The OAuth 2.0 Authorization Framework: Bearer Token Usage](#)
- [OAuth 2.0 — Bearer Tokens](#)

Home Assistant

- [Home Assistant — site oficial](#)
- [Home Assistant — Voice control \(Assist\)](#)
- [Home Assistant — Conversation \(integração\)](#)
- [Home Assistant Developer Docs — Conversation API](#)
- [Home Assistant — Authentication \(Long-Lived Access Tokens\)](#)
- [Home Assistant Developer Docs — Authentication API](#)

Reconhecimento de voz e wake word

- [Vosk — site oficial \(AlphaCephei\)](#)
- [Vosk — modelos disponíveis](#)
- [alphacep/vosk-api \(GitHub\)](#)
- [Picovoice — site oficial](#)
- [Picovoice — documentação do Porcupine](#)

Repositório do projeto

- github.com/Beto-Inovacao-LTDA/jarbas